

Гайд по System Design и Architecture интервью для Senior/Staff Android-инженера

TL;DR. System design / architecture раунд для мобильного инженера оценивает не «идеальное» решение, а сигналы: как вы уточняете требования, ведёте диалог, обосновываете trade-offs и учитываете mobile-специфику (offline, lifecycle, battery, память). Как формулирует Alex Lementuev в «A Simple Framework For Mobile System Design Interviews» (ProAndroidDev/droidcon), «no one expects you to build a robust system in just 30-40 min — the interviewer is mostly looking for specific "signals" from your thought process and communication». Держите в голове один каркас: requirements → scope → high-level → API → data & sync → deep dives → trade-offs → wrap-up, и всегда давайте «default choice + когда отклоняться». Для Android дефолты почти всегда сводятся к слоистой UDF-архитектуре с Room как single source of truth, offline-first через repository + WorkManager, Paging 3 с RemoteMediator, и осознанному выбору транспорта (REST по умолчанию, WebSocket только там, где нужен настоящий real-time).

Раздел 1. Как устроены эти раунды

Формат и тайминг

Каноничный тайминг мобильного system design интервью (по фреймворку Alex Lementuev / weeeBox, разобранному на примере «Design Twitter Feed») на 45-60 минут: 2-5 мин introductions, ~5 мин task definition и requirement gathering, ~10 мин high-level discussion, 20-30 мин detailed discussion (deep dives), ~5 мин вопросы кандидата. Тайминг подтверждается и другими практическими гайдами: по themobileinterview.com («Cracking the Mobile System Design Interview»), «Most system design interviews last around 45 minutes (sometimes they might be extended up to 1 hour)... 6-8 min: Introductions... In reality... you will have around 30 minutes to design your solution». Часть компаний проводит на whiteboard/Excalidraw, часть — в CodeSignal/HackerRank. Код обычно писать не просят — фокус на архитектуре и решениях; иногда попросят набросать псевдокод или структуру данных. ([github](#))

Что реально оценивают (rubric)

Универсальные оси рубрики (по данным Hello Interview, DesignGurus, ByteByteGo Mobile System Design):

- **Requirements Gathering (10-15%)** — правильные вопросы, корректный scoping. Слишком мало вопросов = шаблонность; слишком много = затягивание. ([Substack](#))
- **High-Level Architecture (20-25%)** — правильные компоненты и data flow.
- **Deep dives / Technical Excellence** — глубина по 2-3 компонентам.
- **Trade-offs** — примерно пятая часть оценки уходит именно на явное проговаривание trade-offs.
- **Communication & Collaboration** — умение вести диалог, реагировать на подсказки.

Важная деталь 2026 года (DesignGurus, «System Design Interview Guide 2026»): то, что раньше было «бонусом для staff» (monitoring, deployment, rollback, observability), теперь оценивается явно уже на senior — «They are now graded explicitly at senior level». Мобильный аналог — вы обязаны проговорить наблюдаемость (crash-free rate, ANR, метрики sync-лага), rollout через feature flags и стратегию отката.

«Mobile system design» vs «app/platform architecture»

Это два разных, хотя пересекающихся раунда:

- **Mobile system design** («Design Twitter Feed», «Design a chat app») — сквозная задача: клиент + API contract + data/sync, часто с касанием бэкенда. Оценивают mobile-специфику: offline, кэш, пагинация, транспорт, lifecycle.
- **App/platform architecture** — глубже про внутреннее устройство клиента: слои, модуляризация, DI-граф, границы модулей, тестируемость, публичный API библиотеки/SDK. Если задача — спроектировать библиотеку (Image Loader, Analytics SDK), фокус смещается на дизайн публичного API (об этом прямо предупреждает weeeBox-фреймворк).

Как меняются ожидания Senior → Staff

По рубрикам Hello Interview / ByteByteGo:

- **Senior** ведёт requirements с минимальной помощью, сам поднимает NFR (перф, безопасность, масштаб), проектирует компоненты с чёткими API-контрактами, обрабатывает error handling и edge cases без напоминаний. По ByteByteGo: «Senior engineers should naturally consider issues such as error handling, edge cases, and potential failure scenarios without explicit prompting».
- **Staff+** проактивно gathers requirements, ведёт почти всё интервью, проектирует под расширяемость и эволюцию, оценивает бизнес-влияние и риски, сам поднимает сложные темы (offline-first, кросс-платформенность, failure recovery). По Hello Interview, на staff-уровне кандидат «should... lead almost the entire interview».

Staff-экстра для Android:

- **Модуляризация в масштабе.** По официальному гайду Android и бенчмаркам Automattic (Wojtek Zięba, mobile.blog, 8 сентября 2025, «Gradle modularization delivers 3x faster Android builds»): сравнивали Pocket Casts Android в модульной архитектуре (37 модулей) против монолита (1 модуль) — «modularization delivers 2-3.6x faster build times for local development and 3x faster CI builds. The trade-off? IDE sync times are 5x slower with modules» (бенчмарк через Gradle Profiler на MacBook Pro M1 Max 32GB, задача `assembleDebug`). Паттерн `:feature:x:api` + `:feature:x:impl` для контрактов между командами.
- **Build times.** Gradle параллелит и кэширует по модулям; менять один feature-модуль → пересобирается только он и зависимые.
- **Migration strategy.** Strangler-подход, поэтапная миграция (например, View → Compose), совместимость на границах.
- **ADR/RFC и design docs.** По Martin Fowler и Catio («ADRs: The 2026 Guide»): RFC предлагает (mutable); ADR фиксирует решение (immutable после accept); design doc описывает систему. ADR — одна страница, одно решение, с обязательной секцией «alternatives considered»; решение «architecturally significant», если его дорого откатить, оно пересекает границы команд или ограничивает будущие решения.
- **Cross-team API governance.** Версионирование контрактов, обратная совместимость, владение модулями.

Блок: как отвечать на бэкэнд-вопрос будучи мобильщиком

Ожидаемая глубина: покажите высокоуровневое понимание (load balancer, API gateway, очереди, репликация, fan-out on read vs write) без претензии на экспертизу. Честно обозначьте границы: скоуп мобильного интервью — чаще всего Client-side + API (проектируем клиент и контракт), реже Client + API + Backend (weeeBox прямо отмечает: «Be transparent if your backend experience is limited»). Прямая фраза: *«Мой backend-опыт ограничен; я спроектирую клиент и API-контракт детально, а бэкэнд опишу на уровне интерфейсов и допущений — поправьте, если хотите глубже»*. Честность бьёт блеф — как формулирует newsletter.systemdesign.one: «"I haven't built a system that merges offline edits before, but I'd start with last-write-wins and ask about requirements," is fine. Honesty beats bullshitting».

Раздел 2. Пошаговый фреймворк на 45-60 минут

Бюджет времени (для 45 мин): requirements 6-8 мин → high-level 8-10 мин → API + data/sync 10-12 мин → deep dives 12-15 мин → trade-offs + wrap-up 5 мин.

Шаг 0. Introduction (1-2 мин)

Коротко: стек, роль, масштаб проектов. Не растекаться — каждая лишняя минута отнимается у дизайна.

«Я Android-инженер, Kotlin/Compose/MVI, работал с multimodule-приложением и офлайн-синхронизацией. Веду system design и менторю».

Шаг 1. Requirements clarification (6–8 мин)

Разделите на **Functional** / **Non-Functional** / **Out of Scope**. Выберите 3–5 ключевых функций. Функциональные вопросы: какие экраны/флоу, нужна ли авторизация, pre-fetch. NFR-вопросы: DAU и рост, регионы (emerging markets → размер APK через app bundles, дорогой трафик → минимизировать запросы и агрессивно кэшировать), offline-требования, latency-таргеты, консистентность (eventual vs strong), размер команды (→ модуляризация и структура проекта).

«Прежде чем проектировать — нужен ли offline для этого кейса вообще, или pull-to-refresh достаточно для v1?» «Можем ли мы терпеть eventual consistency для лайков/комментов, или нужна strong?»

Записывайте решения: constraints, must-have vs nice-to-have, open questions.

Шаг 2. Scoping (1–2 мин)

Явно сузьте задачу до решаемой за 40 минут и подтвердите с интервьюером.

«Итого: проектируем media-heavy feed с базовым offline для уже загруженного, кэшируем изображения, prefetch вперёд, pull-to-refresh для v1. Верно?»

Шаг 3. High-level design (8–10 мин)

Спросите, хочет ли интервьюер диаграмму. Дайте BFS-обзор блоков (по weeeBox): API Service (Retrofit) → Repository (медиатор между сетью и хранилищем) → Persistence (Room, SSOT) → UI flows (Compose + ViewModel/MVI) → DI Graph (Dagger) → Image Loader → Analytics Service. Отметьте модульность и параллельную работу команд.

«Начну с большой картины через BFS, потом нырнём туда, куда вам интереснее».

Шаг 4. API contract (5–6 мин)

Установите контракт между клиентом и бэкендом. Для списков — обязательно пагинация (без неё один запрос может вернуть огромный объём, перегружая сеть и память). По умолчанию REST + cursor-based пагинация. Опишите эндпоинты, формат, коды ошибок, идиempотентность для мутаций, exponential backoff и rate limiting.

«Для ленты — cursor-based: `GET /feed?cursor=<opaque>&limit=20`, потому что offset деградирует на больших смещениях (БД скипает offset строк) и ломается при вставках».

Шаг 5. Data & sync (6–8 мин)

SSOT = локальная БД (Room). По официальному гайду Android («Build an offline-first app»): «The local data source is the canonical source of truth for the app... should be the exclusive source of any data that higher layers of the app read». UI всегда читает из Room; сеть — механизм синхронизации. Read-path: repository сразу отдаёт локальное (даже слегка устаревшее), в фоне рефрешит и пишет в Room, Flow автоматически эмитит. Write-path: пишем локально + флаг pending, WorkManager для доставки. Conflict resolution: last-write-wins по timestamp по умолчанию, versioning при необходимости.

Шаг 6. Deep dives (12-15 мин)

Интервьюер выберет 2-3 компонента. Идите вглубь: архитектурный паттерн (MVI/UDF), пагинация (Paging 3 + RemoteMediator), image loading (memory/disk LRU), sync и conflict resolution, транспорт. Проактивно:

▮ *«Прежде чем идти дальше — разберу failure-сценарий для этого компонента».*

Шаг 7. Trade-offs (в процессе и в конце)

Каждое решение — с обратной стороной. Не оставляйте «it depends»: давайте дефолт + условие отклонения.

▮ *«Беру WebSocket ради двусторонней доставки, ценой сложности инфраструктуры и sticky sessions. Если бы это были только серверные уведомления < 1/мин — взял бы SSE».*

Шаг 8. Wrap-up (2-3 мин)

Резюме, известные ограничения, что бы сделали дальше, как мониторили бы в проде.

Раздел 3. Ловушки и анти-паттерны (топ-12)

#	Ошибка	Как выглядит в комнате	Как чинить
1	Прыжок к решению	Начинает рисовать компоненты, не задав ни одного вопроса	2-3 high-impact вопроса до дизайна; проговорить scope
2	Нет уточняющих вопросов	Принимает фичи как данность	Разделить Functional/NFR/Out-of-scope вслух
3	Overengineering	Добавляет ViewModel, DI, кастомные абстракции «на всякий» до обоснования (mobilesystemdesign.com: «makes it seem like you're copying patterns, not solving problems»)	Сначала минимально жизнеспособный дизайн, слои — по мере необходимости
4	Игнор NFR	Проектирует под постоянную сеть	Явно: offline, battery, память, latency-таргеты
5	Нет trade-offs	«Используем Kafka» без «почему»	Каждое решение: «выбираю X ради Y, ценой Z»
6	Игнор подсказок интервьюера	Продолжает свою линию, когда его перебивают	Остановиться, спросить, куда углубиться
7	Молчание при размышлении	Долгие паузы без озвучивания	Думать вслух; проговаривать альтернативы
8	Hand-waving по сложному	Замалчивает error handling, sync, кэш	Именно туда идти вглубь — это и есть сигнал
9	Дизайн только под сегодня	Не думает об эволюции/переиспользовании/модульности	Сказать, как решение растёт под нагрузкой/командой
10	Слишком много про UI	Долго про экраны и пиксели	Держать фокус на data flow и архитектуре
11	Мобилка как изолированный остров	Игнорит API, rate limits, контракты	Явно проектировать API contract и границы
12	Мобильный дизайн как бэкенд	Проектирует под бесконечные ресурсы (systemdesign.one: «Interviewers immediately sense when you're designing for infinite resources instead of real constraints»)	Учитывать реальные ограничения устройства

Отдельно (weeeBox, common-interview-mistakes): не спорьте с интервьюером и не будьте навязчивым — это плохой «сигнал». Если застряли — не сдавайтесь, разбейте задачу на части и покажите процесс.

Раздел 4. Junior vs Senior vs Staff по 8 классическим темам

1. Offline-first sync & conflict resolution

Junior	Senior	Staff
Кэширует ответ в память; сеть = источник правды	Room как SSOT; write locally + pending + WorkManager; last-write-wins с timestamp	Проектирует versioning/vector clocks при необходимости, delta-sync, идемпотентные API, наблюдаемость sync-лага, миграции при offline-апгрейде

2. Pagination

Junior	Senior	Staff
Загружает всё сразу / offset+limit	Paging 3 + PagingSource; cursor-based; понимает деградацию offset	RemoteMediator + Room как кэш, RemoteKeys, стабильность при refresh, prefetchDistance-тюнинг, merge вместо wipe

3. Image loading / caching

Junior	Senior	Staff
Grid грузит полные картинки	Coil/Glide, 3-уровневый кэш (memory LRU → disk → network), downsampling, placeholder	Единый образ размеров для cache-hit, bitmap pooling, реакция на ComponentCallbacks2 / memory pressure, метрики load-source

4. Real-time transport

Junior	Senior	Staff
Polling	Выбирает WebSocket/SSE/push по кейсу; heartbeat, reconnect с backoff	Гибрид: WS в foreground, push в background, resume-токены/seq для догона; sticky sessions, наблюдаемость p95 latency

5. Auth & token refresh

Junior	Senior	Staff
Хранит токен в SharedPreferences	EncryptedSharedPreferences; OkHttp Authenticator на 401; Mutex против гонок refresh	Device-bound refresh (Keystore + подпись challenge), ротация refresh-токенов, proactive refresh по exp, единый logout при протухании

6. Modularization & DI

Junior	Senior	Staff
Всё в app-модуле	Feature/core/data модули, Dagger-граф, чёткие границы	api/impl-паттерн, контроль гранулярности, build-time бюджеты, governance контрактов между командами

7. Error handling / resilience

Junior	Senior	Staff
try/catch в UI	Data слой ловит технические ошибки → domain-ошибки; retry с backoff	Единая таксономия ошибок, circuit breaker, graceful degradation, наблюдаемость и алертинг

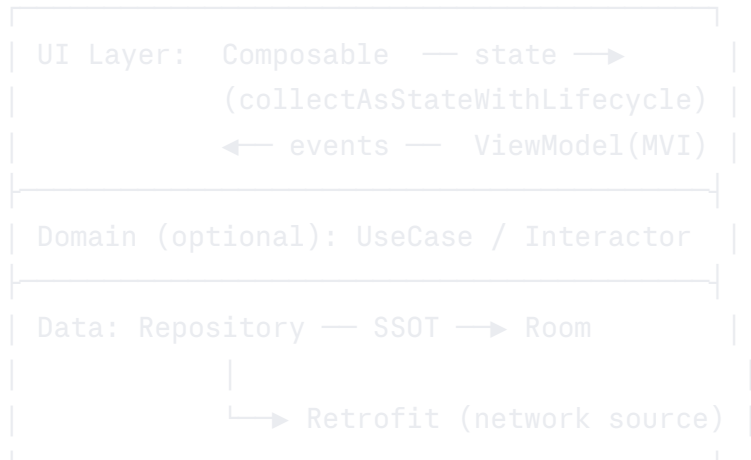
8. Analytics & experiments

Junior	Senior	Staff
Логирует эвенты напрямую	Абстракция Analytics, батчинг эвентов, единая taxonomy	Feature flags как kill switch, экспозиция экспериментов со стабильным hash-based bucketing, эвент на каждую flag-evaluation, приватность/консистентность данных

Раздел 5. Референсные архитектуры («default + когда отклоняться»)

5.1. Каноничная Android UDF-архитектура

Default: слоистая архитектура (официальный гайд Android «Guide to app architecture») — UI layer (Compose + ViewModel/state holder) → (опционально) Domain layer (use cases) → Data layer (repositories + data sources). State течёт вниз, events — вверх (UDF). SSOT в data-слое. «State flows down and events flow up».



Когда отклоняться: для маленьких экранов domain-слой опустить (лишняя сложность — официальный гайд: «for small apps or logic used by a single screen, adding a domain layer may add unnecessary complexity»). Для очень простого приложения UI может дергать repository напрямую, но для масштабируемого — только через ViewModel/domain.

Kotlin, MVI-контур:

kotlin

```

data class FeedUiState(
    val items: List<Post> = emptyList(),
    val isLoading: Boolean = false,
    val error: UiError? = null,
)

class FeedViewModel(private val repo: FeedRepository) : ViewModel() {
    private val _state = MutableStateFlow(FeedUiState())
    val state: StateFlow<FeedUiState> = _state.asStateFlow()

    fun onIntent(intent: FeedIntent) = when (intent) {
        FeedIntent.Refresh -> refresh()
        is FeedIntent.Like -> like(intent.id)
    }
    // state вниз, intents вверх
}
  
```

Правило границ repository: suspend-функции для разовых действий (login, submit), `Flow<T>` для непрерывных потоков (live-список); никогда не отдавать наружу mutable-данные или сырые network/DB-модели.

5.2. Offline-first с SSOT

Default: Room — единственный источник, UI читает Flow из Room. Read-path: repository сразу отдаёт локальное, в фоне рефрешит и пишет в Room, Flow автоматически эмитит. Write-path: пишем локально + флаг pending, WorkManager гарантирует доставку, при успехе снимаем флаг, при конфликте — стратегия резолва. Conflict resolution по официальному гайду: «last write wins» — устройства прикладывают timestamp-метаданные, сервер отбрасывает более старые записи. **Когда отклоняться:** stock trading / live-аукционы (нужна серверная правда в реальном времени), полностью read-only контент (хватит простого кэша), жёсткие ограничения хранилища на very low-end устройствах. (Medium)

5.3. Клиент-серверные паттерны

Транспорт	Default-кейс	Когда отклоняться
REST	CRUD, ленты, большинство запросов	Нужен стриминг/малый оверхед → gRPC
gRPC	Низкая latency, эффективный бинарный трафик, мобильные	Ограниченная поддержка в браузере для двусторонних клиентских кейсов
WebSocket	Двусторонний real-time (чат, presence, collaborative editing)	Оверкилл для редких серверных уведомлений
SSE	Однонаправленный server→client стрим (< 1/ мин), авто-reconnect с Last-Event-ID	Нужна двусторонность или бинарь (SSE — только текст)
Push (FCM/ APNs)	Достучаться до offline/закрытого приложения	Не для гарантированной мгновенной доставки активному пользователю

Гибрид (рекомендация для чата, подтверждается практикой Meta Messenger): WebSocket в foreground + push для offline-соя; при возврате в foreground догон через resume-токены/sequence numbers.

5.4. Кэширование и синхронизация

- TTL** — время жизни кэша; refresh не слишком часто (трафик/батарея), не слишком редко (staleness).
- ETag / conditional GET** — (If-None-Match) → (304 Not Modified), экономит трафик.
- Delta sync** — тянуть только изменения с курсора/версии, не весь набор. **Default:** SSOT-кэш в Room + navigation-based pull sync для коротких офлайнов; delta-sync для больших наборов.

5.5. Offset vs cursor pagination

	Offset	Cursor
Запрос	<code>?offset=100&limit=20</code>	<code>?cursor=<opaque>&limit=20</code>
Плюс	Прыжки на страницу, простота	Стабильна при вставках, быстрая на больших данных
Минус	Деградация на больших offset (БД скипает N строк), дубли/пропуски при вставках	Нет прыжков на произвольную страницу

Default: cursor-based для лент и бесконечного скролла.

Offset — только для небольших списков с нумерацией страниц.

5.6. Background work (WorkManager)

Default: WorkManager для гарантированной, откладываемой работы (sync, upload логов/аналитики, периодический sync) — «work that is required to run reliably even if the user navigates off a screen, the app exits, or the device restarts» (официальный гайд). Constraints (network/battery/charging), exponential backoff, `enqueueUniqueWork`/`enqueueUniquePeriodicWork` против дублей, expedited work для важного пользовательского действия (отправка сообщения/картинки). **Ограничения:** по официальным докам Android («Define work requests») «The minimum repeat interval that can be defined is 15 minutes (same as the JobScheduler API)» — более короткие интервалы WorkManager молча зажимает до `MIN_PERIODIC_INTERVAL_MILLIS`. Не для точного тайминга (→ AlarmManager) и не для real-time / in-process работы (→ Coroutines/foreground service).

kotlin

```
val sync = OneTimeWorkRequestBuilder<SyncWorker>()
    .setConstraints(Constraints.Builder()
        .setRequiredNetworkType(NetworkType.CONNECTED).build())
    .setBackoffCriteria(BackoffPolicy.EXPONENTIAL,
        OneTimeWorkRequest.MIN_BACKOFF_MILLIS, TimeUnit.MILLISECONDS)
    .build()
WorkManager.getInstance(ctx)
    .enqueueUniqueWork("sync", ExistingWorkPolicy.KEEP, sync)
```

5.7. Security basics


```
WS → send:      {op:"send", clientMsgId:<uuid>, convId, text, ts}
WS ← ack:       {op:"ack", clientMsgId, serverMsgId, seq}
WS ← message:   {op:"msg", serverMsgId, seq, convId, senderId, text, ts}
WS ← receipt:   {op:"receipt", serverMsgId, state:"delivered|read", userId}
REST GET /conversations/{id}/messages?cursor=<seq>&limit=50    // догон истории
REST GET /sync?since=<seq>                                     // gap recovery
```

Data model (Room):

```
kotlin

@Entity data class MessageEntity(
    @PrimaryKey val clientMsgId: String,    // UUID, генерится клиентом
    val serverMsgId: Long?,                // назначает сервер
    val seq: Long?,                        // монотонный порядок в диалоге
    val convId: String,
    val senderId: String,
    val text: String,
    val ts: Long,
    val status: String,    // SENDING, SENT, DELIVERED, READ, FAILED
)

@Entity data class OutboxEntity(           // очередь на отправку
    @PrimaryKey val clientMsgId: String, val convId: String, val payload: String,
    val attempts: Int = 0
)
```

Client architecture: UI (Compose/MVI) → ChatViewModel → ChatRepository (SSOT) → Room + WebSocket client + REST + Outbox Worker. Отправка: пишем в `messages` со статусом SENDING и в `outbox` → optimistic UI показывает сообщение сразу → WS шлёт с `clientMsgId` → по `ack` обновляем `serverMsgId`/`seq`, статус SENT, чистим outbox. Если WS недоступен — Worker досылает при появлении сети.

Deep dive 1 — Offline queue и exactly-once display. Клиент генерирует `clientMsgId` (UUID) и использует его как idempotency key. Этот паттерн — прямой аналог Telegram-овского `random_id`: `messages.sendMessage` принимает `random_id: long`, и при потере ответа сервер через `updateMessageID` возвращает связку `random_id → message id`, чтобы клиент ассоциировал ранее отправленное сообщение с доставленным (Telegram API, «Client-side optimisation»). Так даже при повторной пересылке дубль не показывается. Outbox в Room переживает рестарты/краши.

Deep dive 2 — Sync и gap detection. Каждому сообщению/событию присваивается монотонно растущий `seq` (в реальных системах: Telegram использует `pts`/`qts`/`seq`, Discord Gateway — поле `s`: «the sequence number of the event... the relative order in which it occurred», Slack RTM — per-connection integer id). При реконнекте клиент знает свой последний `seq` и запрашивает `/sync?since=<seq>` (аналог Discord Opcode 6 Resume, который «will respond by replaying all missed events in order», и Telegram `updates.getDifference`). Если разрыв слишком большой — полный ресинк диалога (Telegram `differenceTooLong`). Формулировка цели прямо из Telegram API: «to guarantee that no update is missed or applied twice».

Deep dive 3 — Delivery statuses. Стейт-машина: SENDING → SENT (сервер принял, ack) → DELIVERED (доставлено на устройство получателя) → READ — прямой аналог модели WhatsApp (одна серая галочка = отправлено на сервер, две серые = доставлено, две синие = прочитано). Receipts текут отдельными WS-сообщениями. Для offline-получателя сообщение персистится на сервере, шлётся push, а receipt DELIVERED ждёт реального появления клиента (getstream.io: «Delivery receipt waits until Bob's client actually comes back»). Ключевой архитектурный принцип: «every message has a durable home in the message store before the system tries to deliver it».

Trade-off таблица:

Решение	Плюс	Минус	Когда иначе
WebSocket	Двусторонний real-time	Sticky sessions, сложность, хрупкость на моб. сети	Только серверные уведомления → SSE
clientMsgId дедуп	Exactly-once display	Доп. поле/бухгалтерия	—
Room outbox	Переживает краши	Больше кода	Только online-чат → in-memory
last-write-wins	Просто	Возможна потеря правок	Совместное редактирование → CRDT

5 вероятных follow-up:

1. *Как гарантировать порядок в группе?* — Серверный per-conversation `seq`; клиент сортирует по `seq`, буферизует до заполнения `gap`.
2. *Что при отправке трёх сообщений подряд оффлайн?* — Все в `outbox` с отдельными `clientMsgId`, `Worker` досылает по порядку, `exponential backoff` при ошибках.
3. *Multi-device read state?* — Fan-out receipt на все устройства пользователя, cross-device sync читаемости по `serverMsgId` (паттерн: «Fan out to all three chat servers... track per-device delivery»).
4. *Дубли при реконнекте?* — Дедуп по `serverMsgId/clientMsgId` при возврате в `foreground`.
5. *Как масштабировать на большие группы?* — Fan-out на бэкенде через `durable queue` (Kafka, партиция на диалог для `ordering`); клиент это не решает, но должен переживать всплески через `sync-cursor`.

Задача B. Infinite media feed (pagination, prefetch, image pipeline, offline cache)

Уточняющие вопросы: Типы медиа (фото/видео)? Нужен ли `offline` для просмотренного? Live-обновления или `pull-to-refresh`? Размер страницы? Emerging markets (трафик/размер)? Ранжирование на сервере или клиенте?

Допущения: фото (+ короткие видео опционально), базовый `offline` для загруженного, `pull-to-refresh v1`, страница 20, ранжирование на сервере, `cursor-based`.

High-level (ASCII):



API contract:

```
GET /feed?cursor=<opaque>&limit=20
→ { items:[{id, authorId, imageUrl, w, h, blurhash, createdAt}], nextCursor }
GET /feed (If-None-Match: <etag>) → 304 если не изменилось
```

Data model (Room):

kotlin

```
@Entity data class FeedItem(  
    @PrimaryKey val id: String, val authorId: String,  
    val imageUrl: String, val width: Int, val height: Int,  
    val blurhash: String?, val createdAt: Long  
)  
  
@Entity data class RemoteKeys(  
    @PrimaryKey val itemId: String, val prevKey: String?, val nextKey: String?  
)
```

Client architecture: Paging 3 — `Pager(config, remoteMediator = FeedRemoteMediator, pagingSourceFactory = { dao.pagingSource() })`. По официальному гайду Android («Page from network and database»): RemoteMediator «acts as a signal from the Paging library when the app has run out of cached data», тянет из сети и пишет в Room; Room (через сгенерированный PagingSource) отдаёт данные в UI — «the app uses the database as the source of truth... only displays data that has been cached in the database». При refresh — не прыгать всегда на page 1, а использовать RemoteKeys ближайшего к позиции элемента. (Substack)

Deep dive 1 — Pagination с RemoteMediator. `PagingConfig(pageSize=20, prefetchDistance=10)`. RemoteKeys хранят prev/next курсоры в Room → пагинация стабильна после рестарта/refresh (Room инвалидирует запросы при вставке, создавая новый PagingSource с обновлёнными данными). Важно: при загрузке page 1 не вайпать весь кэш — merge с существующим (иначе мигание и потеря офлайн-данных). (Substack)

Deep dive 2 — Image pipeline. Coil (Kotlin/Coroutines/Compose-friendly) с 3-уровневым кэшем: memory LRU → disk (OkHttp/DiskLruCache) → network. По официальным докам Coil (coil-kt.github.io) дефолтный memory cache настроен как `MemoryCache.Builder().maxSizePercent(context, 0.25)` — то есть ~25% доступной памяти приложения; disk cache по умолчанию `maxSizePercent(0.02)` (~2% доступного места). (Для сравнения: старый Android-паттерн ручного `LruCache` для bitmap рекомендовал ~1/8 heap.) Downsampling под размер вью, единый URL/размер для cache-hit, placeholder из blurhash, prefetch картинок на N элементов вперёд под скролл. Реакция на memory pressure через ComponentCallbacks2.

Deep dive 3 — Offline cache и prefetch. Room хранит метаданные ленты (доступны офлайн), Coil disk-кэш хранит байты картинок. Prefetch: и данные (prefetchDistance), и изображения (предзагрузка вперёд). Для emerging markets — webp/AVIF, агрессивный disk-кэш thumbnails, стрим полного медиа только online. (DroidCon)

Trade-off таблица:

Решение	Плюс	Минус	Когда иначе
Paging 3 + RemoteMediator	Offline + сеть из коробки	RemoteKeys-бойлерплейт	Только сеть → чистый PagingSource
cursor pagination	Стабильна при вставках	Нет прыжков на страницу	Нумерованные страницы → offset
Coil 3-tier cache	Быстрый скролл	Память под bitmap	Very low-end → уменьшить memory cache %
Room как SSOT	Мгновенный старт офлайн	Больше кода	Read-only без офлайна → простой кэш

5 вероятных follow-up:

1. *Как избежать *jank* при скролле?* — Downsampling, prefetch картинок, стабильные ключи в LazyColumn, не декодить на main thread, lifecycle-aware Coil-запросы.
2. *Дубли/пропуски при обновлении ленты?* — Cursor + merge в Room по PK; не полагаться на offset.
3. *Как показать свежесть без полной перезагрузки?* — ETag/conditional GET + delta по cursor; pull-to-refresh обновляет верх.
4. *Память при длинном скролле?* — Paging сбрасывает далёкие страницы (maxSize в PagingConfig), Coil эвиктит по ComponentCallbacks2.
5. *Видео в ленте?* — Отдельный pipeline: HLS/DASH стриминг, preload manifest, автоплей только видимого, отдельный бюджет кэша.

Раздел 7. Wrap-up

30-минутный pre-interview cheat sheet

1. **Каркас (2 мин чтения):** requirements → scope → high-level → API → data/sync → deep dives → trade-offs → wrap-up.
2. **Всегда разделяй:** Functional / NFR / Out-of-scope; выбери 3-5 фич.
3. **Mobile NFR-чеклист:** offline, lifecycle, память, батарея, трафик, размер APK, emerging markets, latency, консистентность.
4. **Дефолты Android:** UDF + Room SSOT + repository; Paging 3 + RemoteMediator; Coil 3-tier (~25% памяти memory-кэш); REST+cursor; WebSocket только для real-time; WorkManager для sync (мин. period 15 мин); EncryptedSharedPreferences + cert pinning (2 ключа).
5. **Транспорт-решётка:** REST (дефолт) / gRPC (low-latency, бинарь) / WS (двусторонний) / SSE (одностор., авто-reconnect) / push (offline reach).
6. **Всегда:** «default choice + когда отклоняться», проговаривай trade-off вслух («X ради Y ценой Z»).
7. **Staff-триггеры:** модуляризация api/impl (2-3.6x быстрее сборка, но 5x медленнее IDE sync), build times, ADR/RFC/design doc, migration strategy, cross-team API governance, observability (crash-free, ANR, sync-лаг), rollout через feature flags + rollback.
8. **Поведение:** думай вслух, задавай вопросы, реагируй на подсказки, будь честен о границах, не спорь, не сдавайся при затыке.
9. **Backend-вопрос:** высокоуровнево (LB, gateway, очереди, fan-out) + честные границы + фокус на клиент/API.
10. **Wrap-up:** резюме + ограничения + мониторинг + next steps.

15 быстрых Q&A для самопроверки

1. **С чего начинать?** — С уточнения требований (Functional/NFR/Out-of-scope), не с решения.
2. **Offset или cursor?** — Cursor по умолчанию (стабильность при вставках, перф на больших данных).
3. **Кто SSOT в offline-first?** — Локальная БД (Room); сеть — механизм синхронизации.
4. **Как отправлять сообщения оффлайн?** — Записать локально + outbox + WorkManager, optimistic UI.
5. **Как дедуплировать сообщения?** — clientMsgId (UUID) как idempotency key (аналог Telegram (`random_id`)).
6. **Как догнать пропущенное после реконнекта?** — Монотонный seq/cursor + sync since=<seq> (Discord Resume / Telegram getDifference).
7. **WebSocket vs SSE?** — WS двусторонний; SSE одностор. + авто-reconnect с Last-Event-ID; для чата WS(+push).
8. **Как чинить гонку refresh-токенов?** — Mutex/OkHttp Authenticator: только один refresh реально уходит, остальные ждут и переиспользуют результат.
9. **Где хранить токен?** — EncryptedSharedPreferences; high-value → device-bound через Keystore + подпись challenge.
10. **Минимальный период WorkManager?** — 15 минут; не для точного тайминга/real-time.
11. **Как ускорить сборку в масштабе?** — Модуляризация (параллелизм Gradle, кэш по модулям), api/impl-паттерн.
12. **Дефолт по image loading?** — Coil/Glide, 3-tier LRU кэш, downsampling, prefetch, единый URL для cache-hit.
13. **Conflict resolution по умолчанию?** — Last-write-wins по timestamp; versioning/CRDT при необходимости.
14. **Что оценивают в first order?** — Requirements gathering, high-level architecture, deep dives, trade-offs, communication.
15. **Как отвечать «it depends»?** — Никогда не оставлять так: дать дефолт + чёткое условие отклонения.

Caveats (об источниках)

- **Рубрики и проценты весов** (10–15% requirements, 20–25% architecture) — обобщение из публичных материалов Hello Interview, DesignGurus, ByteByteGo; официальные внутренние рубрики Google/Meta закрыты, реальные веса варьируются по компаниям и интервьюерам. weeeBox прямо отмечает: «This guide does not reflect the interviewing policies from Google (or any other company)».
- **Real-world messenger-механизмы** (pts/qts/seq, `random_id`, Discord `s`/Resume, WhatsApp-галочки) взяты из первичных источников: Telegram Core API (core.telegram.org/api/updates, [/api/optimisation](https://core.telegram.org/api/optimisation)), Discord Developer Docs, Slack docs, WhatsApp Security Whitepaper. WhatsApp/Signal не публикуют детали внутреннего протокола доставки-квитанций — модель галочек описана в consumer-facing Help Center; Slack RTM официально помечен как legacy/deprecated (актуально — Events API / Socket Mode).
- **Coil-дефолты кэша** (25% памяти / 2% диска) — из официальной документации Coil; конкретные значения могут меняться между мажорными версиями библиотеки.
- **Бенчмарк модуляризации** (2–3.6x / 3x / 5x) специфичен для Pocket Casts на конкретном железе (Automattic, сентябрь 2025) — иллюстративен, не универсальная константа; выигрыш зависит от структуры графа зависимостей.
- **Тренд 2026** об оценке observability/rollback уже на senior — из DesignGurus; трактуйте как сильную рекомендацию проговаривать эти темы, а не как жёсткое правило всех компаний.